SAP ABAP

Training Program for Engineering Freshers

SESSION 3

ABAP Data Dictionary (DDIC)

Domains • Data Elements • Tables • Structures • Views • Search Help

Session Overview

DDIC (Data Dictionary) is the backbone of every SAP system. Every table, every field, every data type that exists in SAP is defined here. As an ABAP developer you will open SE11 every single working day — to understand table structures, check field lengths, create your own Z-tables, and build search helps. This session covers it all from scratch to production-level usage.

#	Module	Topics	Duration
1	What is DDIC?	Role, SE11, Repository objects overview	20 min
2	Domains	Definition, value ranges, fixed values	30 min
3	Data Elements	Semantic meaning, search help attachment, F1 help	30 min
4	Database Tables	Transparent/Pooled/Cluster, keys, delivery class	45 min
5	Structures	Include structures, append structures, DDIC vs TYPE	30 min
6	Table Types	TYPE TABLE OF, SORTED, HASHED — in DDIC	20 min
7	Views	Database, Projection, Maintenance, Help views	30 min
8	Search Helps	Elementary, collective, how to attach	25 min
9	Hands-On Lab	Create Domain, Data Element, Z-Table end to end	40 min
10	Recap + Interview Q&A;	Top DDIC interview questions, homework	10 min

TRAINER TIP

Real-world context: In your first week on any SAP project, you will spend 50% of your time in SE11 understanding existing tables before writing a single line of code. This session is project-critical.

Module 1 — What is ABAP Data Dictionary (DDIC)?

The ABAP Data Dictionary is a **central repository** inside SAP that defines all data structures used by ABAP programs. When you create a table in DDIC, it is automatically created in the underlying database (Oracle, HANA, etc.) as well. When you change a field, the change is reflected everywhere in the system.

Key points about DDIC:

- Single source of truth — one change in DDIC propagates to all programs using it.
- Active/Inactive objects — DDIC objects must be activated before use (like ABAP programs).
- Database independent — DDIC abstracts the underlying DB (Oracle, HANA, DB2).
- Fully integrated with ABAP — 'TYPE mara-matnr' directly uses DDIC metadata.

Repository Objects in DDIC (SE11):

Object Type	What It Is	Example
Domain	Defines technical properties: data type, length, value range	CHAR 18 — for material numbers
Data Element	Adds business meaning + field/column labels to a Domain	MATNR — 'Material Number'
Database Table	Physical table stored in DB — rows and columns of business data	MARA — Material Master General
Structure	Like a table but not stored in DB — used in programs/interfaces	BAPIRET2 — BAPI return structure
Table Type	Defines the type of an internal table in DDIC	BAPIRET2_T — table of BAPIRET2
View	Virtual table combining multiple real tables	MARD — Material/Storage Location
Search Help	F4 dropdown/popup for a field	MAT1 — Search help for MATNR
Lock Object	Ensures data consistency when multiple users edit same record	EMARA — Lock for MARA table
Type Group	Groups type definitions (old style, rarely used in modern ABAP)	SYST — SAP system type group

INTERVIEW POINT

DDIC interview question: 'What is the difference between a Domain and a Data Element?' This is asked in almost every ABAP interview. Domain = technical type + length + value range. Data Element = business meaning + labels + points to a Domain. One Domain can be reused by many Data Elements.

Module 2 — Domains

A **Domain** is the lowest level in the DDIC hierarchy. It defines the **technical characteristics** of a field: what type of data it holds, how long it is, and what values are allowed.

What a Domain defines:

Property	Meaning	Example
Data Type	ABAP elementary type (CHAR, NUMC, DEC, DATS, TIMS...)	CHAR
Length	Number of characters or digits	18
Decimal Places	For DEC/CURR/QUAN types only	2
Output Length	Display length in screens/reports	18
Fixed Values	Allowed values list (makes F4 dropdown automatically)	'A'=Active, 'I'=Inactive
Value Table	Reference to a check table for foreign key validation	T001 (Company codes)
Sign	Whether negative values are allowed (for numeric types)	Yes/No
Conversion Exit	Special routine to convert display format	ALPHA — removes leading zeros

Creating a Domain in SE11 — Step by Step:

1. Go to SE11 → Select Domain radio button → Enter name **ZSTATUS_VEN** → Click Create.
2. Enter Short Description: 'Vendor Status Code'
3. On Definition tab: Data Type = CHAR, No. of Characters = 1, Output Length = 1
4. On Value Range tab → Fixed Values → Add: A = Active, I = Inactive, B = Blocked
5. Save (Ctrl+S) → Activate (Ctrl+F3)

TRAINER TIP
After adding Fixed Values to a Domain, any field using this domain automatically gets an F4 value-list dropdown. No extra coding needed — this is the power of DDIC!

Common SAP Standard Domains (memorize these):

Domain	Type	Length	Used For
CHAR1	CHAR	1	Single character flag (X/blank)
MATNR	CHAR	18	Material number
LIFNR	CHAR	10	Vendor number
KUNNR	CHAR	10	Customer number
BUKRS	CHAR	4	Company code
WERKS	CHAR	4	Plant
MENGE	QUAN	13	Quantity (3 decimal places)
DMBTR	CURR	13	Amount in local currency (2 decimal places)
DATUM	DATS	8	Date
UZEIT	TIMS	6	Time

Module 3 — Data Elements

A **Data Element** sits on top of a Domain. It adds **business meaning** to the technical definition. It provides the field labels you see on SAP screens, the F1 help documentation, and links the field to a Search Help.

Domain vs Data Element — The Golden Rule:

	Domain	Data Element
Defines	Technical type + length + allowed values	Business meaning + screen labels
Example	CHAR, length 18	'Material Number', label 'Material'
Reusable?	Yes — many data elements can use one domain	Yes — many table fields can use one data element
Contains labels?	No	Yes — Short, Medium, Long, Heading
F1 Help?	No	Yes — documentation text shown on F1
Search Help?	No — but can have Value Table	Yes — Search Help attached here

Creating a Data Element in SE11 — Step by Step:

1. SE11 → Select Data Type → Enter name **ZVENDOR_STATUS** → Create
2. Choose 'Data element' → Enter description: 'Vendor Status'
3. Type Definition tab: Type Name = ZSTATUS_VEN (our domain from Module 2)
4. Field Label tab: Short=Status, Medium=Vend Status, Long=Vendor Status, Heading=Status
5. Save → Activate

TRAINER TIP

The Field Label tab is very important. The Short label appears next to the field in transactions. The Heading appears as column header in reports. These are maintained ONCE in the Data Element and appear consistently everywhere.

DDIC Hierarchy — How It All Connects:

```
DOMAIN (Technical Base)
  ■■■ DATA ELEMENT (Business Meaning + Labels)
          ■■■ TABLE FIELD (Physical column in a table)
                  ■■■ ABAP VARIABLE  DATA lv_x TYPE <data_element>.

Example:
  Domain:       CHAR, Length 18
  Data Element: MATNR  (label = 'Material Number')
  Table Field:  MARA-MATNR
  ABAP:         DATA lv_matnr TYPE matnr.
```

INTERVIEW POINT

'What is the purpose of a Data Element?' — Gives business meaning + maintains labels centrally. Change the label in the Data Element once, and it updates in ALL transactions and reports automatically. This is SAP's metadata-driven architecture.

Module 4 — Database Tables

Database Tables in DDIC are the most important objects you will work with daily. Every piece of business data in SAP — materials, vendors, sales orders, financial postings — lives in a database table. Understanding table properties is essential for writing correct and performant SELECT statements.

Types of Tables in SAP:

Type	Description	When Used	Example
Transparent Table	1:1 mapping with DB table. Most common type.	All business data storage	MARA, LFA1, KNA1, VBAK
Pooled Table	Multiple DDIC tables stored in one physical DB table.	Old SAP internal config (rare now)	ATAB — Customizing pool
Cluster Table	Multiple DDIC tables stored together + binary compression.	Old FI/HR internal tables (rare now)	PCL1, PCL2 — HR clusters

TRAINER TIP
In modern SAP S/4HANA, pooled and cluster tables are being eliminated. As a fresher, focus on Transparent Tables — that is 99% of your work.

Table Properties — Important Fields in SE11:

Property	Options & Meaning
Delivery Class	A=Application data, C=Customizing, S=System, L=Temporary, G=Customizing (protected)
Data Browser/Table View	Display/Maintenance Allowed — controls SE16/SM30 access
Table Category	Transparent / Pool / Cluster
Key Fields	Fields marked as key — unique identifier of each row (like Primary Key in SQL)
Client Field	MANDT field — makes table client-dependent (separate data per client)
Enhancement Category	Can be enhanced / Must not be enhanced / etc.

Creating a Z Table in SE11 — Step by Step (Full End-to-End):

We will create a table **ZTRAINING_VENDOR** to store training-specific vendor data.

Step 1 — Open SE11, select Database Table, enter ZTRAINING_VENDOR, click Create.

Step 2 — Short Description: 'Training Vendor Master Custom Table'

Step 3 — Delivery Class: A (Application table), Data Browser: Display/Maintenance Allowed

Step 4 — Fields tab — add the following fields:

Field Name	Key	Data Element	Description
MANDT	Yes	MANDT	Client — always first field in client-dependent tables
ZVENID	Yes	LIFNR	Vendor ID — second key field
ZNAME	No	NAME1	Vendor Name
ZCOUNTRY	No	LAND1	Country
ZSTATUS	No	ZVENDOR_STATUS	Vendor Status (our custom data element)
ZCREATED	No	ERDAT	Created On Date

Step 5 — Enhancement Category: Can be Enhanced (under Extras menu)

Step 6 — Save to package $TMP (or your Z-package)

Step 7 — Activate (Ctrl+F3) — this creates the physical table in the database!

Step 8 — Verify in SE16N: open ZTRAINING_VENDOR — it should show empty table

COMMON MISTAKE

MANDT (Client) field must ALWAYS be the first key field in any client-dependent custom table. If you forget MANDT, data will be shared across all clients — a critical mistake in production systems.

Important Standard Tables — Memorize These:

Table	Module	Stores	Key Fields
MARA	MM	Material Master General Data	MANDT, MATNR
MAKT	MM	Material Descriptions (text)	MANDT, MATNR, SPRAS
MARC	MM	Material Data per Plant	MANDT, MATNR, WERKS
LFA1	MM	Vendor Master General Data	MANDT, LIFNR

Table	Module	Stores	Key Fields
LFB1	MM	Vendor Master per Company Code	MANDT, LIFNR, BUKRS
KNA1	SD	Customer Master General Data	MANDT, KUNNR
KNB1	SD	Customer Master per Company Code	MANDT, KUNNR, BUKRS
VBAK	SD	Sales Order Header	MANDT, VBELN
VBAP	SD	Sales Order Items	MANDT, VBELN, POSNR
EKKO	MM	Purchase Order Header	MANDT, EBELN
EKPO	MM	Purchase Order Items	MANDT, EBELN, EBELP
BKPF	FI	Accounting Document Header	MANDT, BUKRS, BELNR, GJAHR
BSEG	FI	Accounting Document Line Items	MANDT, BUKRS, BELNR, GJAHR, BUZEI
T001	FI	Company Codes	MANDT, BUKRS
T001W	MM	Plants	MANDT, WERKS

Module 5 — Structures

A **Structure** in DDIC is like a table but with one critical difference: **it has no data storage**. No physical table is created in the database. Structures are used to define the shape of data that is passed between programs, function modules, and BAPIs.

Table vs Structure:

Feature	Database Table	Structure
Data stored in DB?	Yes — physical table in DB	No — only type definition
Can SELECT from it?	Yes	No
Used as work area?	Yes — LIKE tablename	Yes — main use case
Key fields?	Yes — uniqueness constraint	No keys needed
Examples	MARA, LFA1, VBAK	BAPIRET2, RSDSSELOPT, SYST

Include Structures — Reusing Fields:

DDIC lets you **include** one structure inside another. This is used everywhere in SAP standard — for example, address fields, currency fields, and audit fields are defined once in an include structure and reused across many tables.

```
* Example: ADDR1_DATA include used in multiple tables
* In SE11, a table might look like:

Table: ZVENDOR_FULL
  MANDT    TYPE MANDT         (Key)
  LIFNR    TYPE LIFNR         (Key)
  .INCLUDE  ADDR1_DATA        <-- Includes all address fields at once
  ZSTATUS  TYPE ZVENDOR_STATUS

* This brings in: NAME1, NAME2, STRAS, ORT01, PSTLZ, LAND1, etc.
* No need to define each address field manually!
```

Append Structures — Enhancing Standard Tables:

An **Append Structure** lets you add custom fields to a SAP standard table WITHOUT modifying the standard table itself. This is one of the most important DDIC enhancement techniques.

- Standard table: MARA (Material Master)

- You need to add a custom field Z_CUSTOM_FLAG to MARA
- Solution: Create an Append Structure ZMARA_APPEND with field Z_CUSTOM_FLAG
- Append it to MARA — the field appears in MARA without touching SAP standard

COMMON MISTAKE

Never directly add fields to SAP standard tables (via manual edit). Always use Append Structures. Direct modifications are overwritten during SAP upgrades and cause support pack issues.

INTERVIEW POINT

'What is an Append Structure?' — A DDIC object used to add custom fields to a standard SAP table without modifying it. The append survives SAP upgrades because it is stored separately. This is a very common interview question for ABAP developers.

Module 6 — Table Types

A **Table Type** in DDIC defines the type of an internal table globally — so it can be reused across programs, function modules, and method signatures without redefining it every time.

```
* Without DDIC Table Type — you define it locally every time:
TYPES: BEGIN OF ty_vendor,
         lifnr TYPE lifnr,
         name1 TYPE name1,
       END OF ty_vendor.
DATA lt_vendor TYPE STANDARD TABLE OF ty_vendor.

* WITH DDIC Table Type (e.g., ZVENDOR_TT defined in SE11):
DATA lt_vendor TYPE zvendor_tt.   " Clean, reusable, consistent

* Also used in Function Module / Method parameters:
FUNCTION z_get_vendors.
*   ET_VENDORS    LIKE ZVENDOR_TT  (Export table parameter)
```

Creating a Table Type in SE11:

1. SE11 → Data Type → Enter ZVENDOR_TT → Create → Choose Table Type
2. Line Type: enter the row structure name (e.g., ZTRAINING_VENDOR or a structure)
3. Table Kind: Standard Table / Sorted Table / Hashed Table
4. Key: Empty key (Standard) or specify key fields (Sorted/Hashed)
5. Save and Activate

Table Kind in DDIC	Access Type	Key Unique?	Best For
Standard Table	Index (sequential)	Not required	General output, reports
Sorted Table	Binary search	Optional	Large tables, frequent key reads
Hashed Table	Hash function O(1)	Must be unique	Lookup tables, config data

Module 7 — Views

A **View** in DDIC is a virtual table that presents data from one or more base tables. You can SELECT from a View just like a regular table in ABAP. SAP uses views extensively to simplify complex multi-table data access.

View Type	What It Does	Can Read?	Can Write?	Example
Database View	JOIN of multiple tables at DB level. Most common type.	Yes	No (read-only)	V_MARA — MARA + MAKT joined
Projection View	Subset of columns from ONE table. Hides sensitive fields.	Yes	No	A view of LFA1 hiding bank details
Maintenance View	Used with SM30 to create table maintenance UI for customizing.	Yes	Yes	V_T001 — Company code maintenance
Help View	Used as data source for Search Helps.	Yes	No	Used behind F4 dropdown

Creating a Database View — Step by Step:

Scenario: Create a view that joins MARA (material) with MAKT (description).

1. SE11 → View → Enter ZMAT_DESC_VIEW → Create → Select Database View
2. Short Description: 'Material with Description View'
3. Tables tab: Add MARA and MAKT as base tables
4. Join Conditions tab: MARA-MANDT = MAKT-MANDT and MARA-MATNR = MAKT-MATNR
5. View Fields tab: Select fields you need — MATNR, MBRSH, MTART, MAKTX, SPRAS
6. Selection Conditions tab: SPRAS = 'E' (English descriptions only)
7. Save and Activate

```
* Using the view in ABAP — same as any table:
DATA: lt_mat TYPE STANDARD TABLE OF zmat_desc_view,
      ls_mat TYPE zmat_desc_view.

SELECT matnr maktx mtart
  FROM zmat_desc_view
  INTO TABLE lt_mat
  WHERE mtart = 'FERT'.    " Finished goods only

LOOP AT lt_mat INTO ls_mat.
  WRITE: / ls_mat-matnr, ls_mat-maktx.
ENDLOOP.
```

TRAINER TIP

Database Views are pre-joined at the database level — they are much faster than doing the same JOIN in ABAP. Always prefer a DB View over manual JOINs in ABAP when the join is reused across multiple programs.

Module 8 — Search Helps

A **Search Help** provides the **F4 value popup** for a field. When a user presses F4 on a field (e.g., Material Number), a popup appears with valid values. This is defined in DDIC as a Search Help object — no coding required.

Type	Description	Example
Elementary Search Help	Single search path — one query, one result list	MAT1 — shows list of materials
Collective Search Help	Multiple elementary search helps combined in one F4 popup with tabs	MAT1_WH — materials by different criteria

Creating an Elementary Search Help:

Scenario: F4 help for our ZTRAINING_VENDOR table — user can search vendor by name.

1. SE11 → Search Help → Enter ZVENDOR_SH → Create → Elementary Search Help
2. Selection Method: ZTRAINING_VENDOR (our custom table)
3. Dialog Type: Display values immediately (no dialog before search)
4. Parameters tab: Define which fields are Search/Export/Display:

Parameter	Field	IMP	EXP	LPos	SPos	What it means
ZVENID	ZVENID	X	X	1	1	Import+Export — this is the result field returned
ZNAME	ZNAME			2	2	Displayed in list but not returned
ZCOUNTRY	ZCOUNTRY			3	3	Displayed in list for context

5. Save and Activate the Search Help
6. Attach to Data Element ZVENDOR_STATUS (in Data Element → Search Help tab)

INTERVIEW POINT
'How do you attach a Search Help to a field?' — Three ways: (1) Attach to Domain via Value Table (automatic foreign key check), (2) Attach to Data Element (most common, reusable), (3) Directly in Screen Painter for specific screen fields.

Module 9 — Hands-On Lab: Full DDIC Object Chain

In this lab you will create a complete DDIC object chain from scratch. This is the exact same process you follow on real SAP projects when building custom Z-tables for client requirements.

Objective: Build a Training Employees table — ZTRAINING_EMP

STEP 1 — Create Domain: ZEMP_STATUS

SE11 → Domain → ZEMP_STATUS → Create

Data Type: CHAR, Length: 1, Output Length: 1

Fixed Values: A = Active, I = Inactive, T = Trainee

Save → Activate

STEP 2 — Create Data Element: ZEMP_STATUS_DE

SE11 → Data Type → ZEMP_STATUS_DE → Create → Data Element

Type Name: ZEMP_STATUS (our domain)

Field Labels: Short=Stat, Medium=Emp Status, Long=Employee Status, Heading=Status

Save → Activate

STEP 3 — Create Database Table: ZTRAINING_EMP

SE11 → Database Table → ZTRAINING_EMP → Create

Delivery Class: A, Data Browser: Display/Maintenance Allowed

Add fields as shown below:

Field	Key	Data Element / Type	Description
MANDT	Y	MANDT	Client
ZEMPID	Y	PERSNO	Employee ID
ZFNAME	N	AD_NAMEFIR	First Name
ZLNAME	N	AD_NAMELST	Last Name
ZDEPT	N	ORGEH	Department
ZJOINING	N	DATUM	Date of Joining
ZSALARY	N	DMBTR	Salary
ZSTATUS	N	ZEMP_STATUS_DE	Employee Status (our custom DE)

Enhancement Category: Can be Enhanced (Extras → Enhancement Category)

Save to $TMP → Activate → Verify in SE16N

STEP 4 — Write an ABAP Program to INSERT and READ data

```
REPORT z_emp_lab.

DATA: ls_emp TYPE ztraining_emp.

* Insert a test record
ls_emp-mandt   = sy-mandt.
ls_emp-zempid  = '00000001'.
ls_emp-zfname  = 'Rahul'.
ls_emp-zlname  = 'Kumar'.
ls_emp-zdept   = 'IT'.
ls_emp-zjoining = sy-datum.
ls_emp-zsalary = '50000.00'.
ls_emp-zstatus = 'T'.   " Trainee

INSERT ztraining_emp FROM ls_emp.
IF sy-subrc = 0.
  WRITE: / 'Record inserted successfully'.
ELSE.
  WRITE: / 'Insert failed - record may already exist'.
ENDIF.

* Read all trainees
DATA: lt_emp TYPE STANDARD TABLE OF ztraining_emp,
      lv_count TYPE i.

SELECT * FROM ztraining_emp INTO TABLE lt_emp
  WHERE zstatus = 'T'.

lv_count = lines( lt_emp ).
WRITE: / 'Total Trainees:', lv_count.

LOOP AT lt_emp INTO ls_emp.
  WRITE: / ls_emp-zempid, ls_emp-zfname, ls_emp-zlname.
ENDLOOP.
```

TRAINER TIP
Run this program multiple times — the second run will show sy-subrc <> 0 on INSERT because the primary key already exists. This demonstrates primary key constraint enforcement by DDIC!

Module 10 — Recap, Interview Q&A & Homework

Top DDIC Interview Questions (10 Years of Experience):

Question	Answer in One Line
What is a Domain?	Defines technical type, length, and allowed values of a field
What is a Data Element?	Adds business meaning, screen labels, and F1 help on top of a Domain
Difference between Domain and Data Element?	Domain = technical; Data Element = semantic. One domain, many data elements
What is an Append Structure?	Adds custom fields to a standard SAP table without modifying it
What is a Transparent Table?	1:1 mapping with physical DB table — most common table type in SAP
Why is MANDT always the first key field?	Client dependency — separates data between clients 100, 200, 300
What is a View in DDIC?	Virtual table — joins multiple tables, no physical storage, used for SELECT
What is a Search Help?	Provides F4 dropdown popup for a field — Elementary or Collective
Difference between Structure and Table?	Structure has no DB storage; Table stores physical data in the database
Where do you attach a Search Help?	To the Data Element (recommended), or to Domain, or directly on screen field
What is Enhancement Category?	Property of a DDIC table — controls whether Append Structures are allowed
What is a Table Type in DDIC?	Globally defined internal table type — reused across programs and FMs

What We Covered Today:

- DDIC hierarchy: Domain > Data Element > Table Field > ABAP Variable
- Domain — technical base: type, length, fixed values, value table
- Data Element — business meaning, screen labels, F1 help, search help link
- Database Tables — transparent, delivery class, MANDT key, SE11 creation
- 15 must-know SAP standard tables (MARA, LFA1, KNA1, VBAK, BKPF and more)
- Structures — no DB storage, include structures, append structures for enhancements

• Table Types — globally reusable internal table types
• Views — Database, Projection, Maintenance, Help — SELECT from views in ABAP
• Search Helps — F4 popup: Elementary vs Collective, how to attach
• Full hands-on lab: Domain > Data Element > Table > ABAP program

Homework for Session 4:

1. In SE11, open table LFA1 and note: total fields, key fields, delivery class
2. Open Data Element LIFNR — what Domain does it use? What are the field labels?
3. Create your own Domain ZCOURSE_TYPE (CHAR 2) with fixed values: AB=ABAP, BA=Basis, SD=SD
4. Create Data Element ZCOURSE_TYPE_DE using your domain
5. Create a table ZTRAINING_COURSE with at least 5 meaningful fields
6. Write a SELECT program to read from both ZTRAINING_EMP and ZTRAINING_COURSE

Preview of Session 4:

• Modularization: FORM/ENDFORM, PERFORM, parameter passing
• Function Modules (SE37): CREATE, TEST, parameter types (IMPORT/EXPORT/TABLES/CHANGING)
• ABAP Objects (OOP) Introduction: CLASS, METHOD, CREATE OBJECT
• BAPI concept and how to call standard BAPIs from ABAP
• Exception handling in Function Modules

You now understand the complete DDIC architecture that powers every SAP system. Practise creating these objects until it feels natural — this knowledge is used every single day in real SAP projects.